Task Management API - Django REST Framework

Name:Susheel Kumar Bahuroju g-mail: susheelbahuroju@gmail.com

Project Overview:

This project implements a fully functional Task Management REST API using Django REST Framework.
It supports complete CRUD operations (Create, Read, Update, Delete) with clean and professional code structure.
The API follows REST principles, handles edge cases, validates inputs (like date format), and returns appropriate HTTP status codes with clear messages for both success and error scenarios.
Includes search, date filtering, and ordering features for enhanced usability.

Features

- Add a new task
- Sort tasks by date
- Search tasks by title
- Search tasks by date
- Edit a task (partial update allowed)
- Delete a task
- List all tasks
- Proper error messages for invalid requests (e.g., wrong date format, task not found)

Endpoints Overview

GET /tasks/ - List all tasks with optional search, filter, sort
POST /tasks/ - Create a new task
PATCH /tasks/{id}/ - Update (partial) a task
DELETE /tasks/{id}/ - Delete a task

How to Use Filters

- Search by Title: /tasks/?search=Meeting
- Filter by Date (YYYY-MM-DD): /tasks/?search_date=2024-04-26
- Sort by Last Updated Date: /tasks/?sort_by_date=true

You can combine filters also!
Example: /tasks/?search=Meeting&search_date=2024-04-26&sort_by_date=true

Setup and Installation Steps:

1. Clone the Repository:
   - Start by cloning the repository to your local machine.

2. Set Up a Virtual Environment:
   - python -m venv venv
   - venv\Scripts\activate
   - Ensure python version is minimum 3.6
   - Tested in python 3.11.0 - Working Fine

3. Install Required Dependencies:
   - Ensure you are in django-tasks-assignment directory
   - pip install -r requirements.txt

4. Apply Database Migrations:
   - python manage.py makemigrations
   - python manage.py migrate

5. Run Unit and Integrated testcases (Optional):
   - pytest (Using django-test package)
   - Ensure all test cases are successful before running server

6. Run the Server:
   - python manage.py runserver
   - The API will be accessible at http://127.0.0.1:8000/

7. Import Postman Collection for testing (Optional):
   - A postman collection file is present in repository
   - Import the file into Postman
   - You can directly test CRUD operations from there.

Edge Case Handling

1. Wrong Date Format:
   "Invalid date format. Please use YYYY-MM-DD."

2. No Tasks Found for Search Criteria:
   "No tasks found for date YYYY-MM-DD"
   "No tasks found for title 'task_title'"

3. Empty Request Body for Create/Update:
   Appropriate message indicating that required fields are missing or invalid.

4. Invalid Task ID Format:
   "Invalid task ID format. ID must be numeric."

5. Deleting Already Deleted or Non-Existent Task:
   "Task not found or already deleted."

6. Invalid Sorting Parameter:
   "Invalid value for sort_by_date. It must be 'true'."

Additional Documentation (Commands, Viewsets, Serializers, URL Names)

Commands

- Create virtual environment: python -m venv venv
- Activate virtual environment (Windows): venv\Scripts\activate
- Activate virtual environment (Linux/Mac): source venv/bin/activate
- Install dependencies: pip install -r requirements.txt
- Make migrations: python manage.py makemigrations
- Apply migrations: python manage.py migrate
- Run server: python manage.py runserver
- Run tests (optional): pytest

Viewsets

TaskViewSet:
- list() - List tasks
- create() - Create a new task
- partial_update() - Update task partially (PATCH)

- destroy() - Delete a task
- Validates ID format and empty PATCH body

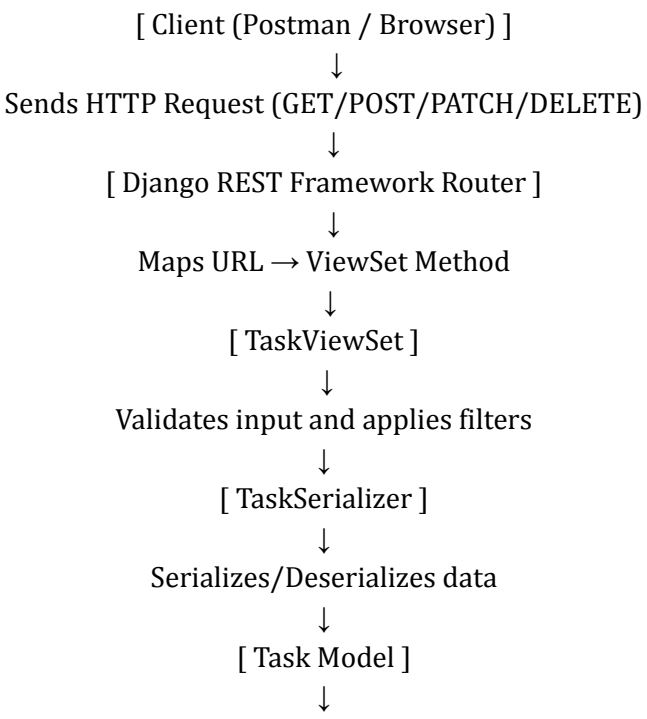
Serializers

TaskSerializer:
- id
- title
- description
- created_at
- updated_at
- completed
- Handles validation automatically

URL Names

- POST /tasks/ - Create task
- GET /tasks/ - List tasks
- PATCH /tasks/{id}/ - Update task
- DELETE /tasks/{id}/ - Delete task

Request-Response Flow



Interacts with Database

↓

[Django ORM]

↓

[Django REST Framework]

↓

Sends HTTP Response (Status Code + JSON)

↓

[Client Receives Response]